Lokakarya

# Coding Python

## Loop & List untuk Coding yang Efisien

Disusun Oleh
Narasumber UPT TIKP



# Tujuan

- Menguasai perulangan: Perulangan While, Do-While, For, Nested, dan Forever
- Mengotomatiskan tugas berulang menggunakan struktur loop
- Bekerja dengan daftar: Pembuatan, akses, pengeditan, pemotongan, dan metode bawaan
- Menulis program interaktif menggunakan loop dan struktur data

# While, Do-While, For, Nested, and Forever Loops in Python

# Perulangan dalam Python

Perulangan sangat penting dalam pemrograman karena membantu mengotomatiskan tugas-tugas yang berulang. Alih-alih menulis kode yang sama berulang kali, perulangan memungkinkan kita mengeksekusi blok kode berulang kali berdasarkan kondisi tertentu. Python mendukung beberapa jenis perulangan, termasuk while, for, perulangan bersarang, dan perulangan tak terbatas (perulangan selamanya).

# Perulangan while

Perulangan `while` mengeksekusi blok kode selama kondisi yang diberikan tetap terpenuhi `True`.

**Sintaksis:**

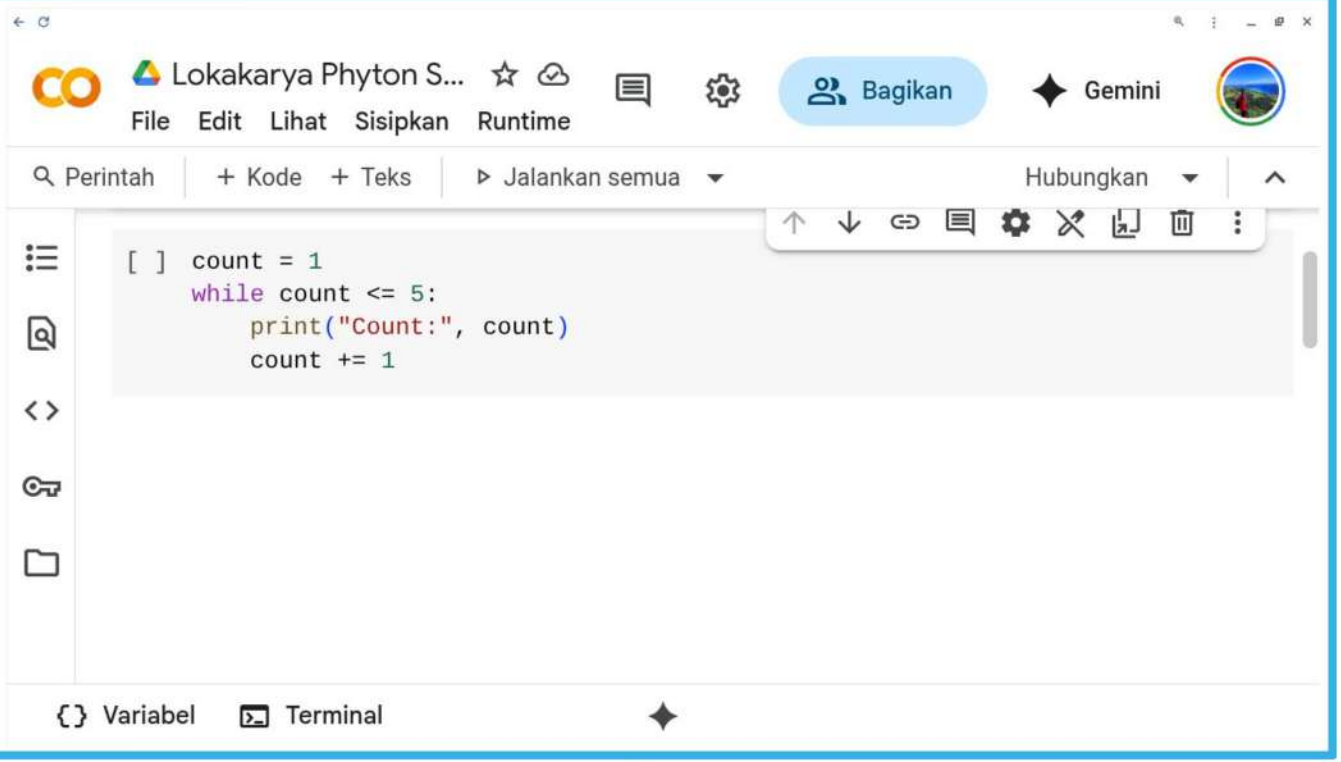
```
while condition:
    # Code to be executed
```

**Contoh:**

```
count = 1
while count <= 5:
    print("Count:", count)
    count += 1
```

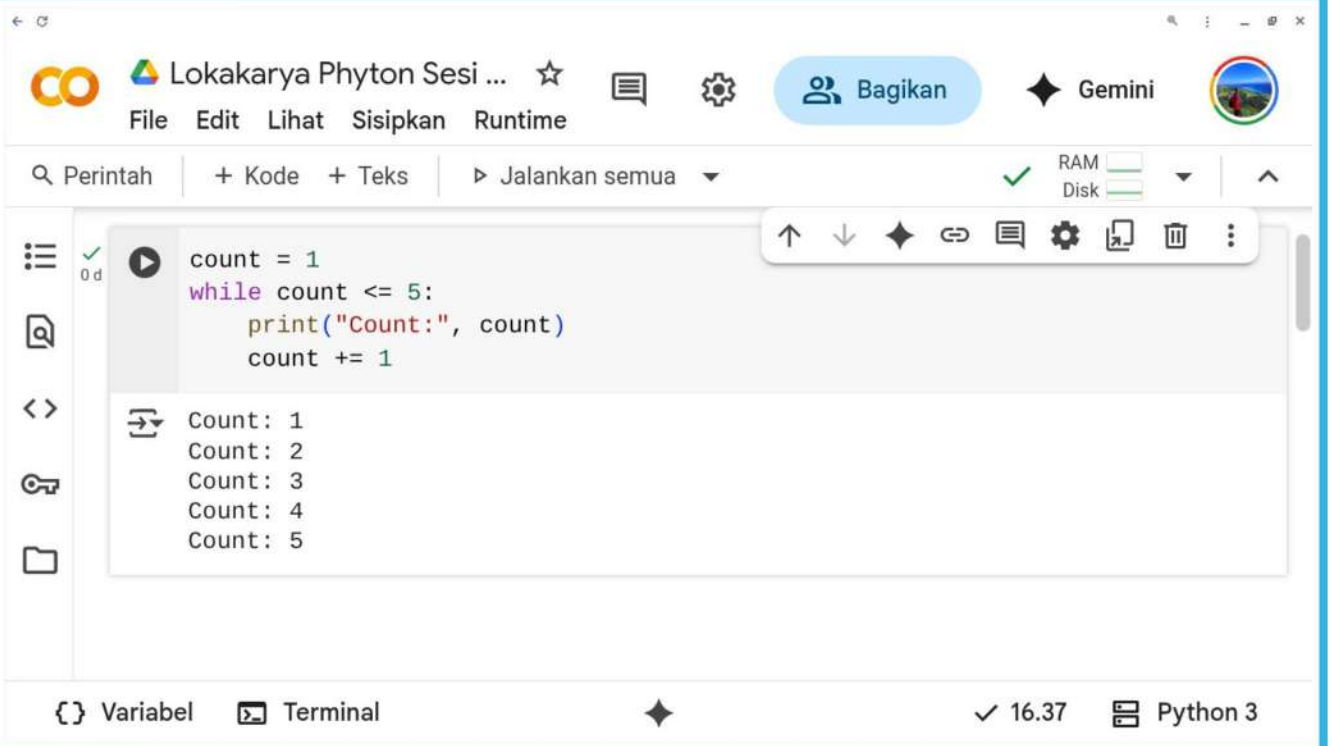
# Perulangan while

Mengulangi hitung sampai sama dengan 5



# Perulangan while

Hasil Menghitung Sampai 5



# Perulangan while

**Penjelasan:**

- Perulangan dimulai dengan `count = 1`.
- Berjalan bila `count` kurang dari atau sama dengan 5.
- Pada tiap iterasi, ia mencetak nilai `count` dan meningkatkannya sebesar 1.
- Jika `count` melebihi 5, putaran berhenti.

# Perulangan Do-While

Python tidak memiliki loop bawaan `do-while`, tetapi dapat disimulasikan menggunakan perulangan `while` dengan `break` pernyataan.

**Sintaksis:**

```
while True:
    # Code to be executed
    if condition:
        break
```

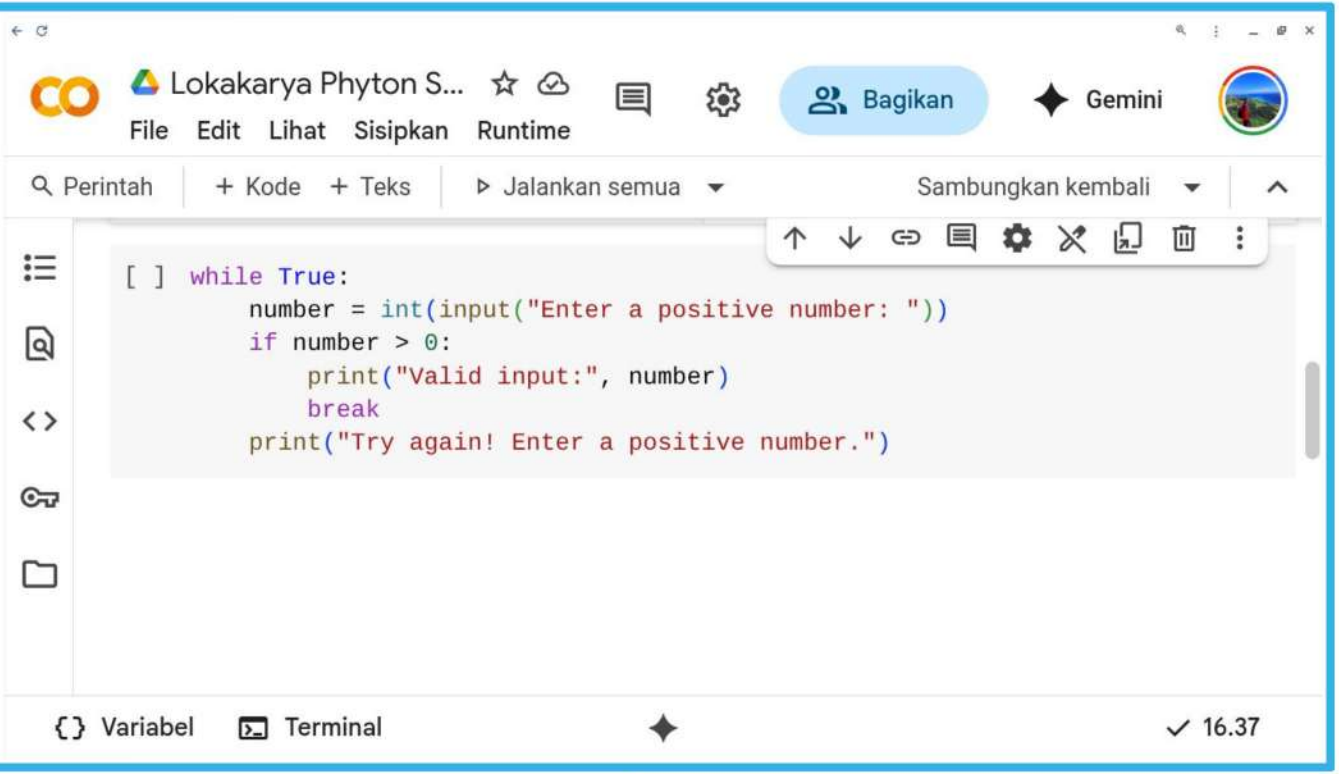
# Perulangan Do-While

**Contoh:**

```
while True:
    number = int(input("Enter a positive number: "))
    if number > 0:
        print("Valid input:", number)
        break
    print("Try again! Enter a positive number.")
```

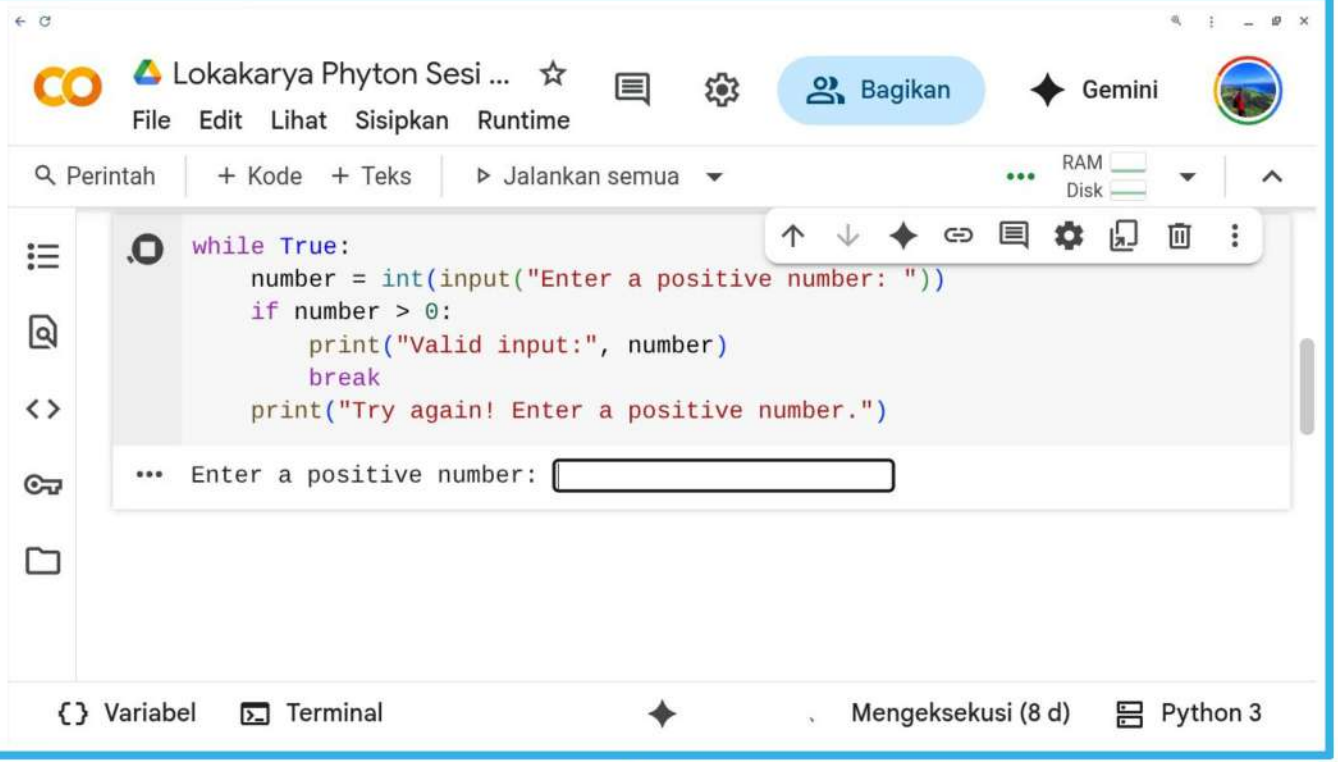
# Perulangan Do-While

Validasi Bilangan Positif



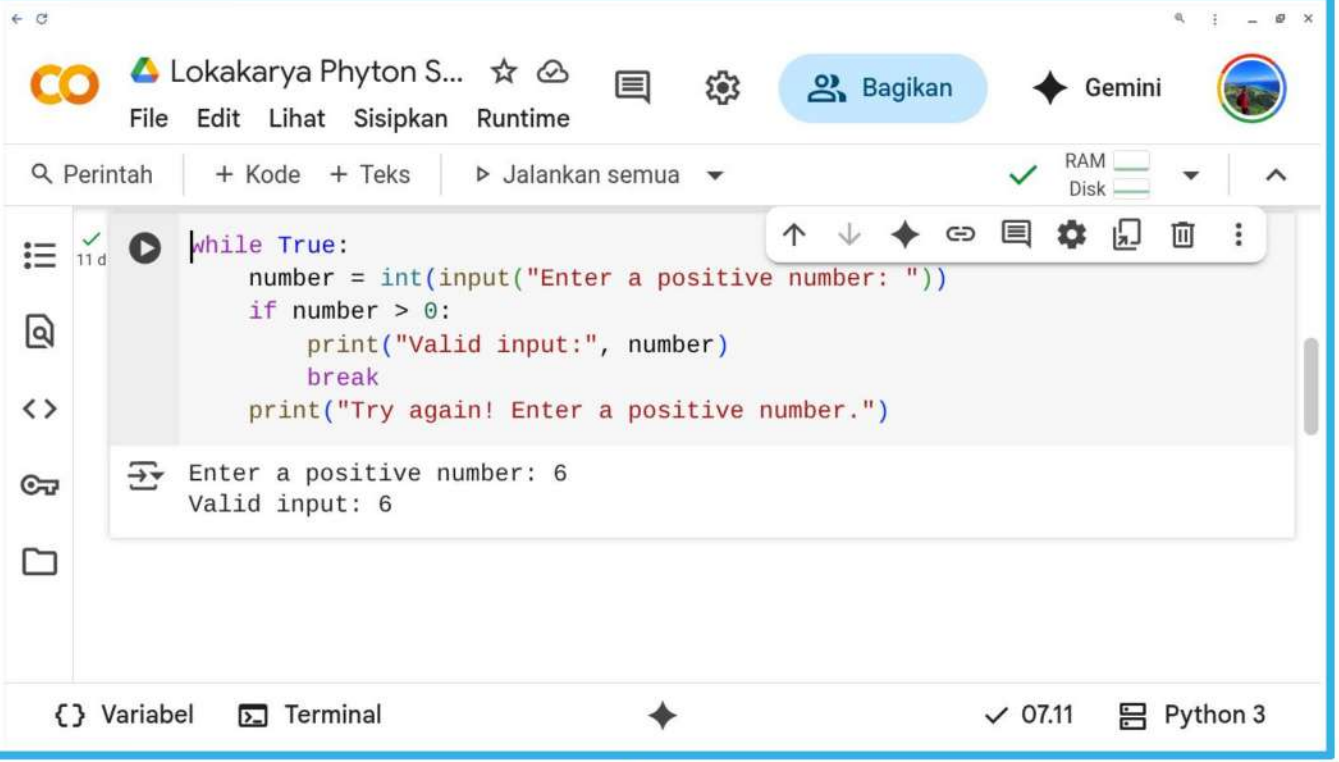
# Perulangan Do-While

Masukkan data angka



# Perulangan Do-While

Hasil validasi angka



# Perulangan Do-While

**Penjelasan:**

- Perulangan berjalan tanpa batas ( `while True`).
- Meminta masukan pengguna.
- Jika angkanya positif, ia mencetak nilai dan keluar dari loop ( `break`).
- Jika tidak, ia akan mengingatkan pengguna lagi.

# Perulangan For

Perulangan `for` dalam Python terutama digunakan untuk mengulang suatu urutan (seperti daftar, string, atau rentang).

**Sintaksis:**

```
for variable in sequence:
    # Code to be executed
```

# Perulangan For

**Contoh:**

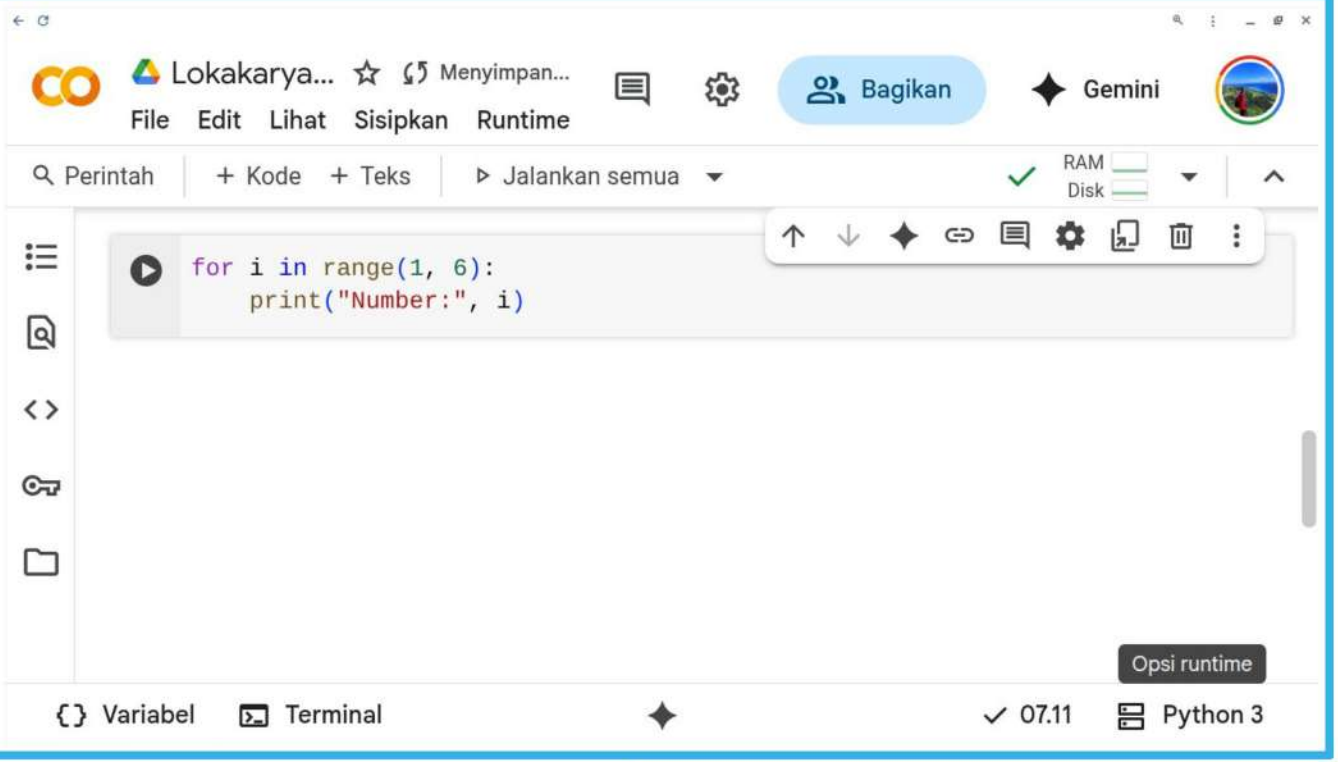
```
for i in range(1, 6):
    print("Number:", i)
```

**Penjelasan:**

- `range(1, 6)` menghasilkan angka dari 1 sampai 5.
- Perulangan berjalan melalui setiap angka dan mencetaknya.

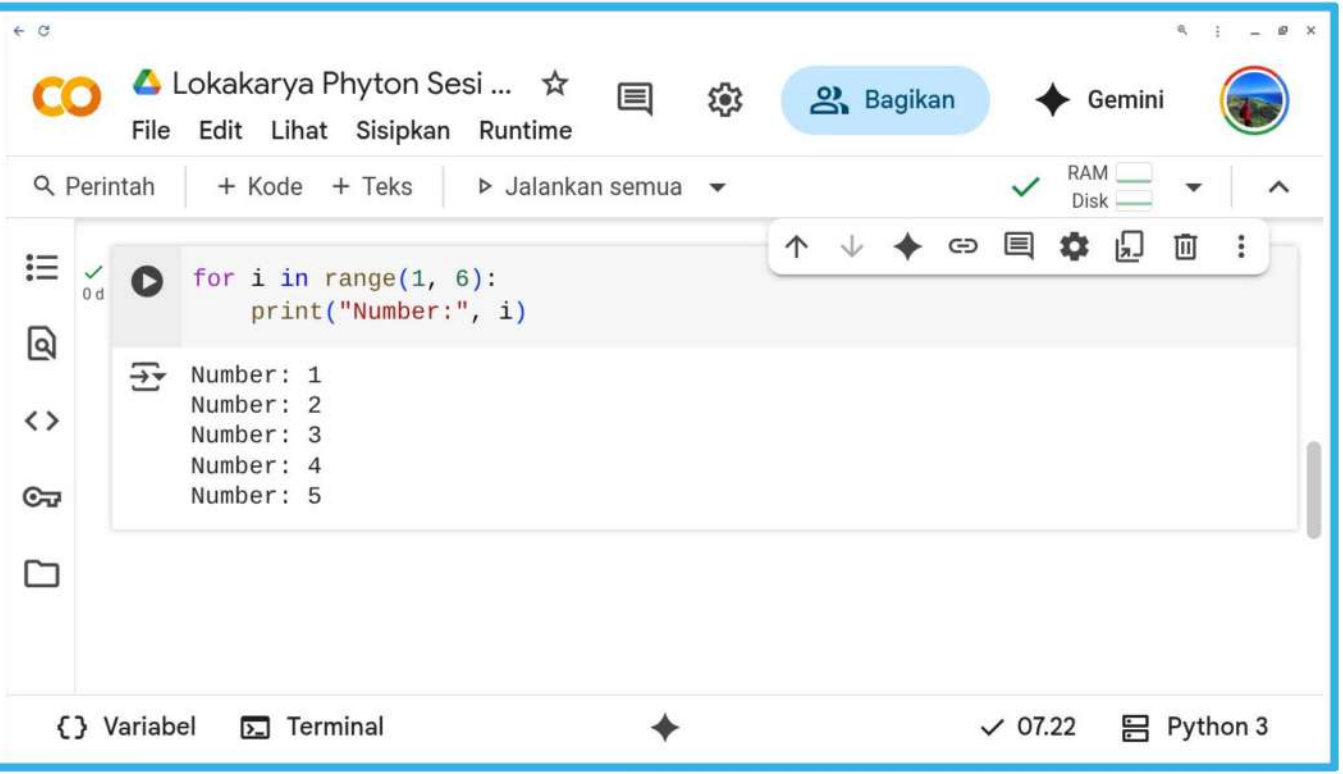
# Perulangan For

Cetak Angka rentang mulai 1 sampai sebelum 6



# Perulangan For

Hasil Cetak Angka rentang mulai 1 sampai sebelum 6



# Loop Bersarang

Perulangan bersarang adalah perulangan di dalam perulangan lain. Setiap iterasi perulangan luar mengeksekusi seluruh perulangan dalam.

**Sintaksis:**

```
for outer_variable in range(n):
    for inner_variable in range(m):
        # Code to be executed
```

# Loop Bersarang

**Contoh:**

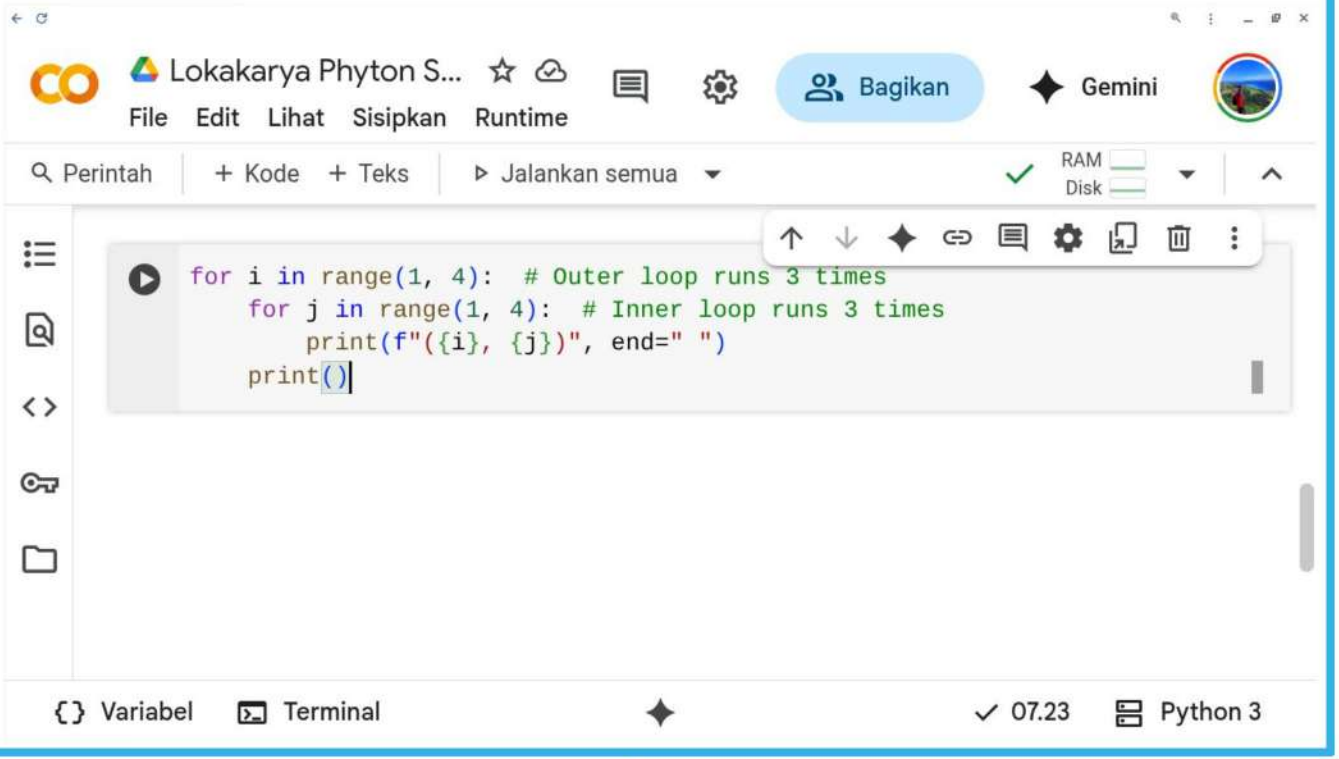
```
for i in range(1, 4):  # Outer loop runs 3 times
    for j in range(1, 4):  # Inner loop runs 3 times
        print(f"({i}, {j})", end=" ")
    print()
```

**Penjelasan:**

- Loop luar berjalan 3 kali ( `i` dari 1 hingga 3).
- Untuk setiap `i`, loop dalam berjalan 3 kali ( `j` dari 1 hingga 3).
- Hasilnya adalah kisi pasangan angka.

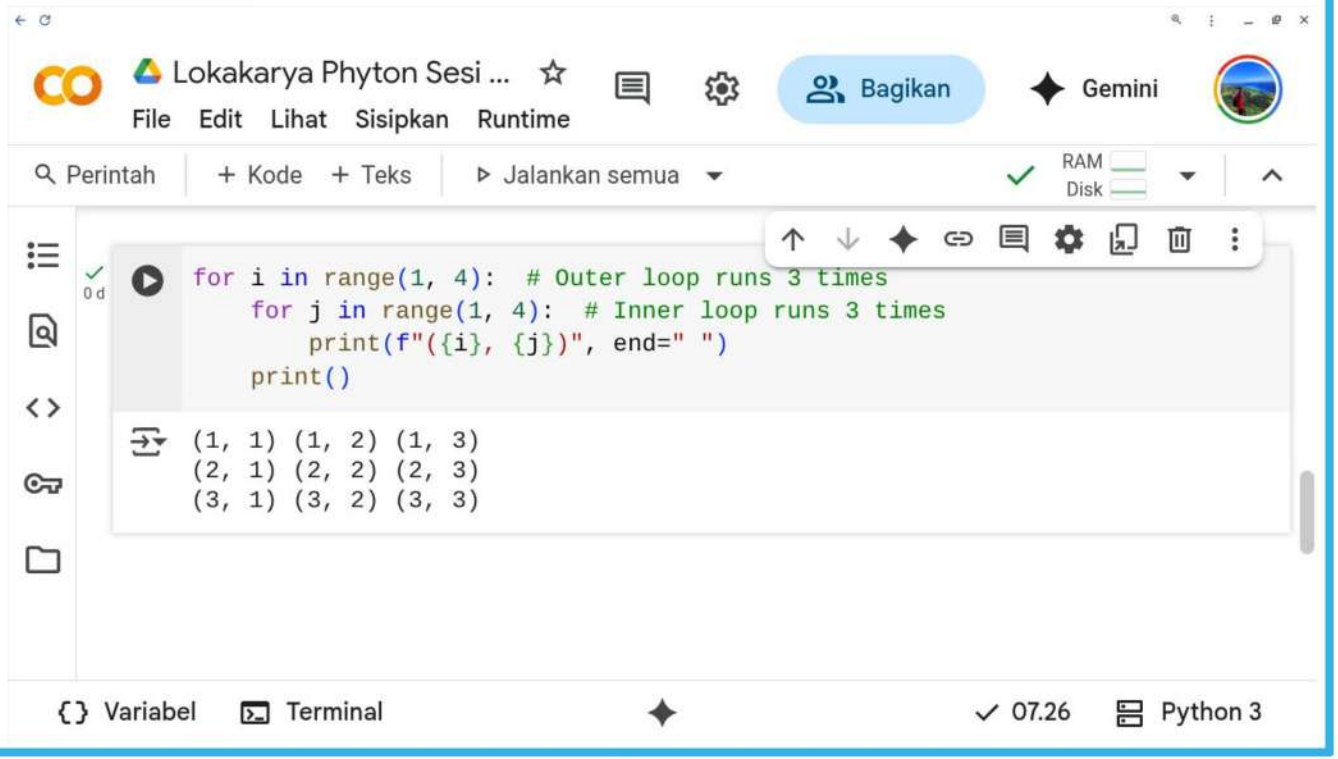
# Perulangan Bersarang

Perulangan yang berulang



# Perulangan Bersarang

Hasil Perulangan yang berulang



# Forever Loop (Loop Tak Terbatas)

Perulangan tak terhingga berjalan tanpa henti kecuali diakhiri dengan menggunakan `break`.

**Sintaksis:**

```
while True:
    # Code to be executed
```

# Forever Loop (Loop Tak Terbatas)

**Contoh:**

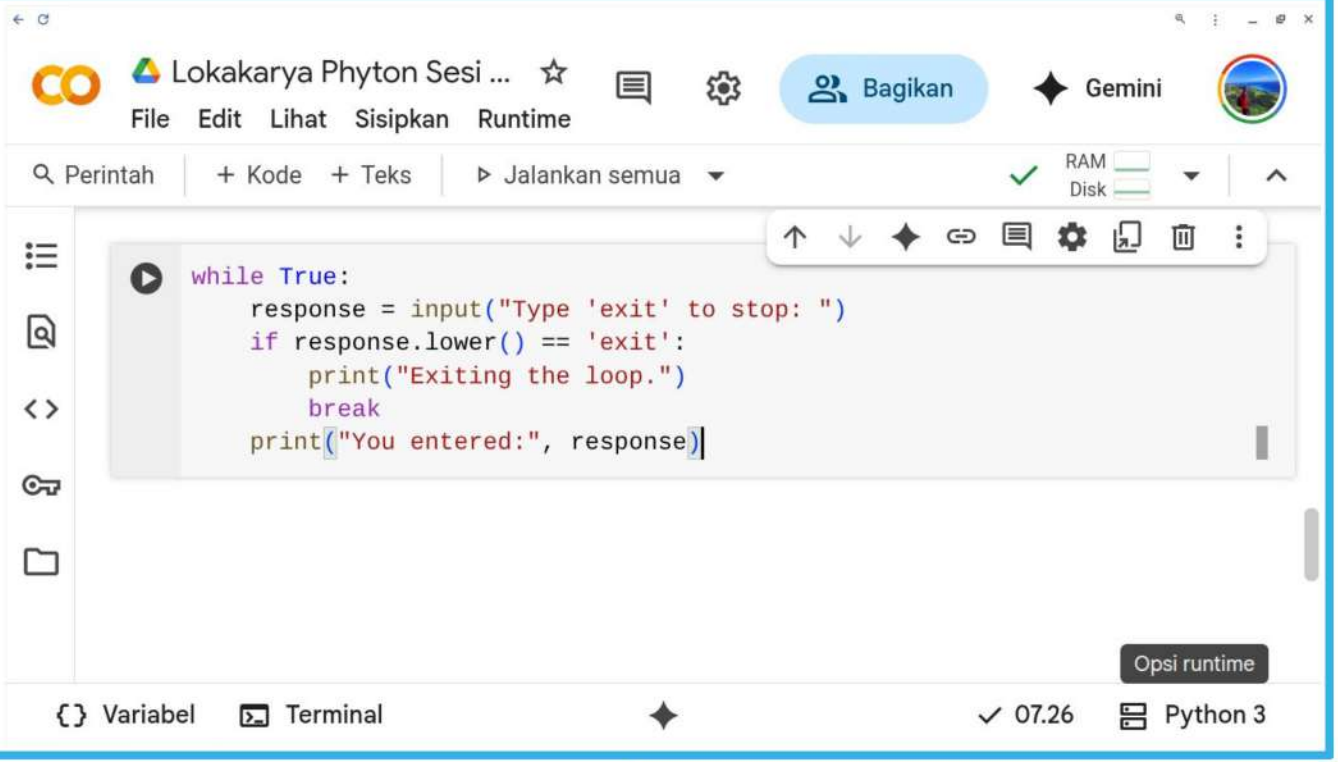
```
while True:
    response = input("Type 'exit' to stop: ")
    if response.lower() == 'exit':
        print("Exiting the loop.")
        break
    print("You entered:", response)
```

**Penjelasan:**

- Perulangan berjalan selamanya ( `while True`).
- Ia mengambil masukan pengguna dan memeriksa apakah itu `'exit'`.
- Jika ya, ia akan keluar dari lingkaran. Jika tidak, ia akan terus berlanjut.

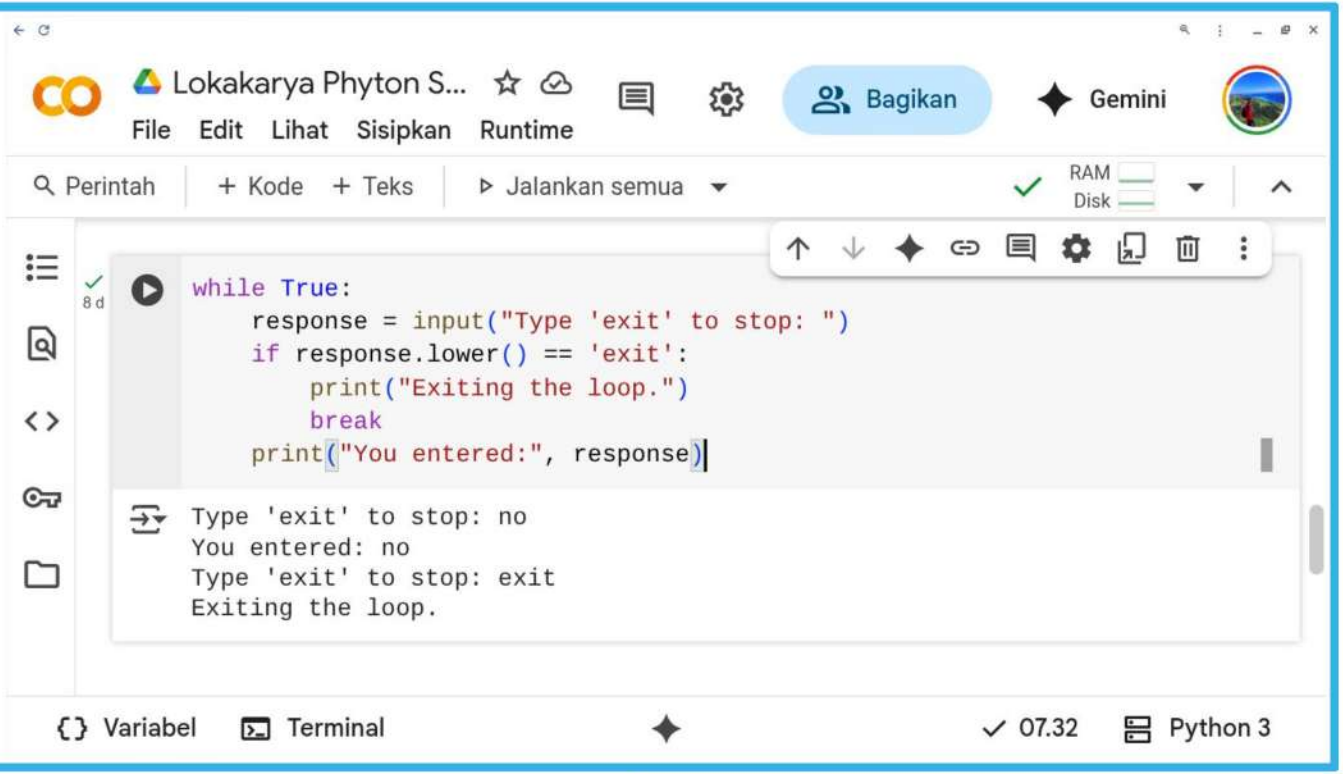
# Perulangan Tak Terbatas

Sebelum syarat Berhenti Dipenuhi, Program terus berjalan



# Perulangan Tak Terbatas

Hasil masukkan syarat berhenti



# Bekerja dengan List

List adalah salah satu struktur data yang paling umum digunakan dalam Python. List memungkinkan Anda menyimpan beberapa nilai dalam satu variabel dan melakukan berbagai operasi seperti mengakses, mengedit, mengiris, dan menggunakan metode bawaan. List bersifat **terurut** , **dapat diubah** , dan **memungkinkan nilai duplikat** .

# Membuat Daftar

Daftar dalam Python dibuat menggunakan tanda kurung siku `[]`, dan elemen-elemennya dipisahkan dengan koma.

**Sintaksis:**

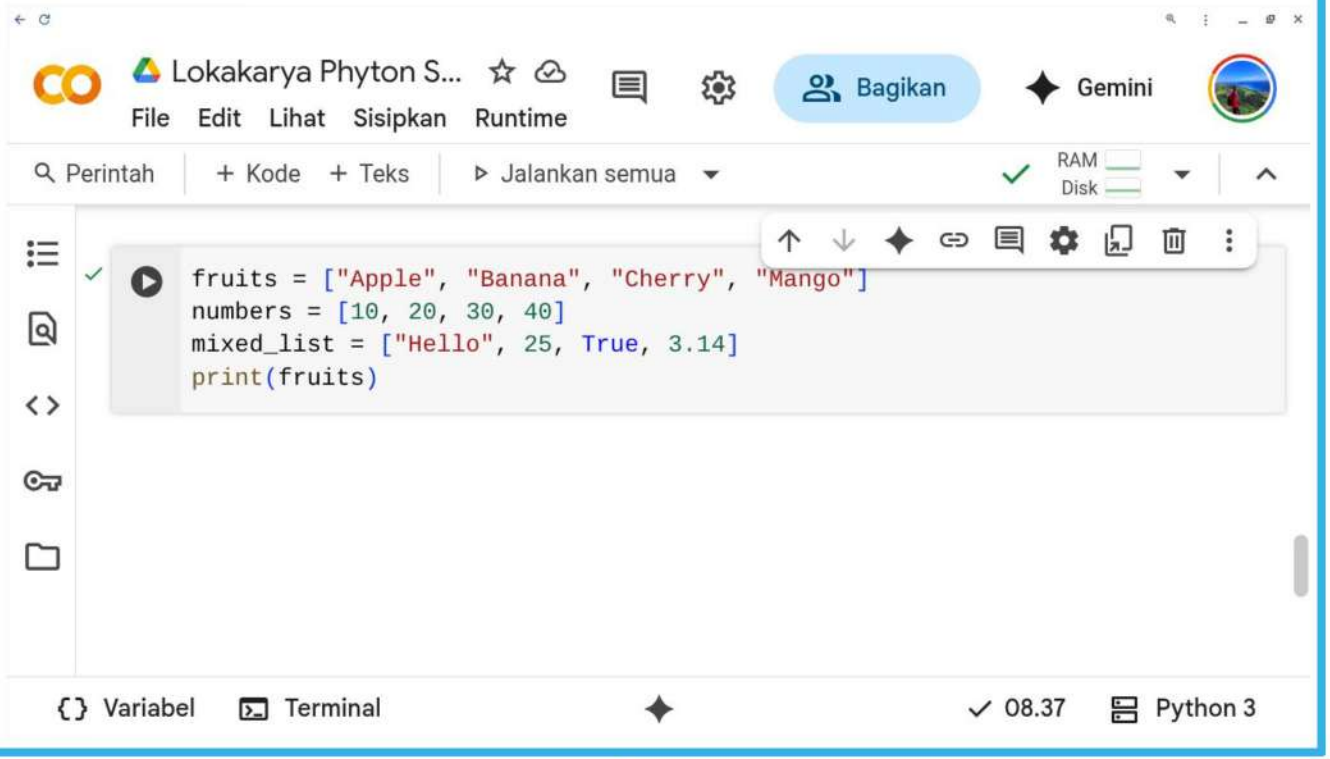
```
list_name = [element1, element2, element3, ...]
```

**Contoh:**

```
fruits = ["Apple", "Banana", "Cherry", "Mango"]
numbers = [10, 20, 30, 40]
mixed_list = ["Hello", 25, True, 3.14]
print(fruits)
```

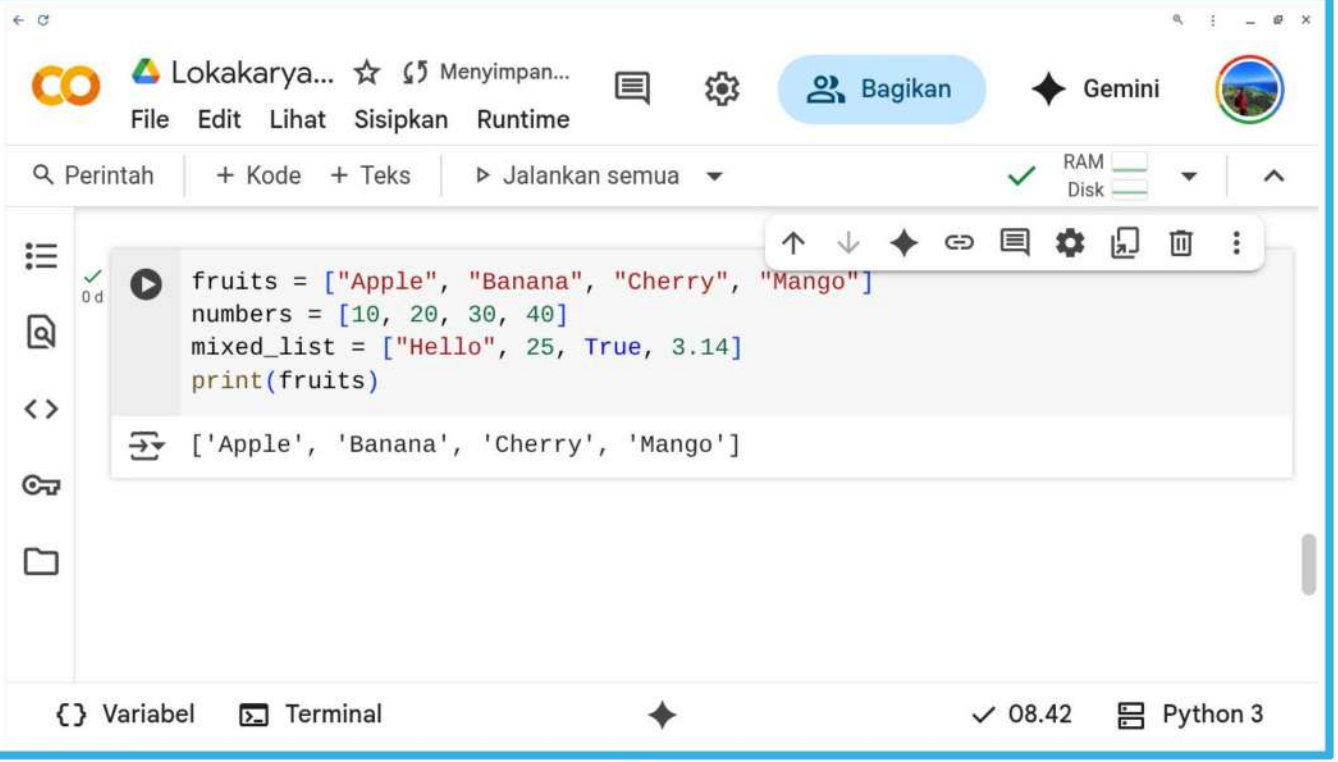
# Membuat Daftar

Membuat daftar buah



# Membuat Daftar

Hasil Membuat daftar buah



# Membuat Daftar

**Penjelasan:**

- `fruits` adalah daftar yang berisi elemen string.
- `numbers` adalah daftar bilangan bulat.
- `mixed_list` berisi berbagai tipe data (string, integer, boolean, float).
- Daftar dapat menampung **semua** jenis data, termasuk daftar lainnya.

# Mengakses Elemen dalam Daftar

Python menggunakan **pengindeksan** untuk mengakses elemen. Pengindeksan dimulai dari **0** untuk elemen pertama. Pengindeksan negatif dimulai dari `-1` untuk elemen terakhir.

**Sintaksis:**

```
list_name[index]
```

# Mengakses Elemen dalam Daftar

**Contoh:**

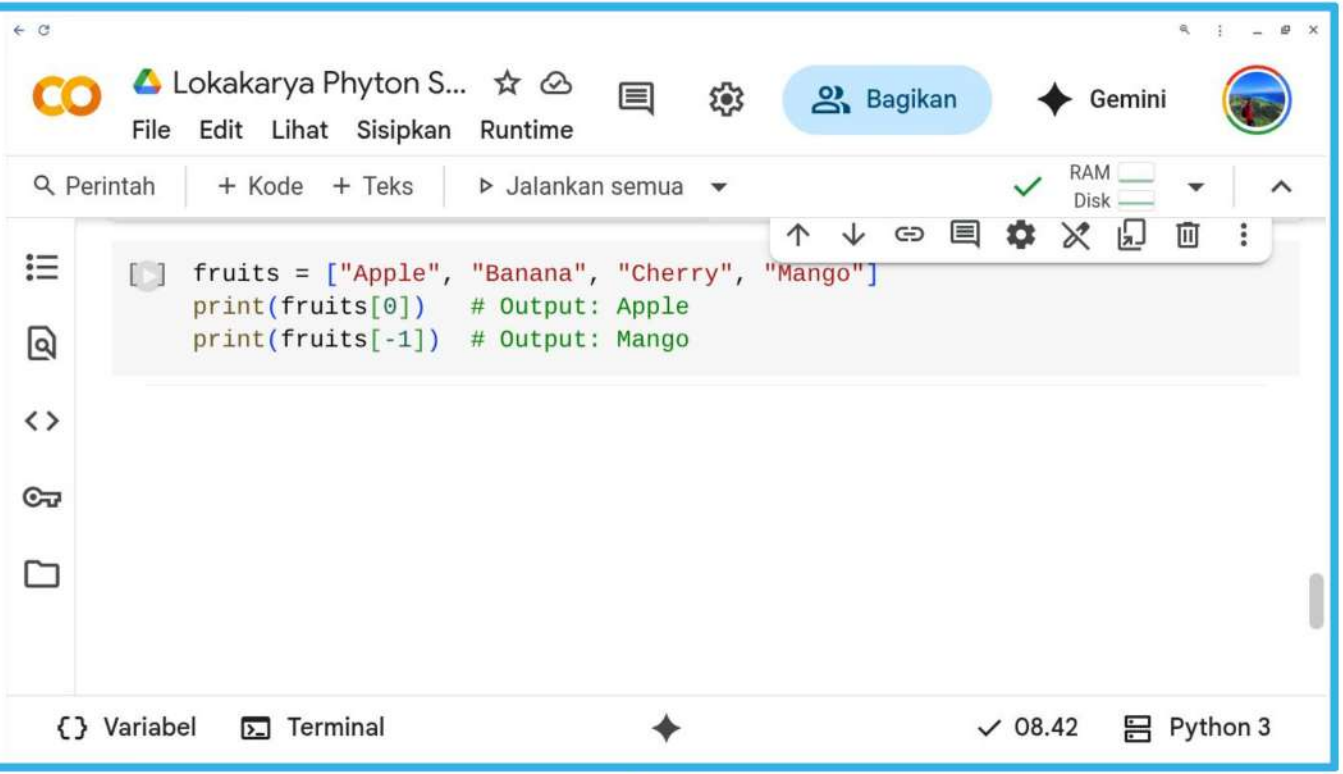
```
fruits = ["Apple", "Banana", "Cherry", "Mango"]
print(fruits[0])    # Output: Apple
print(fruits[-1])   # Output: Mango
```

**Penjelasan:**

- `fruits[0]` memberikan elemen pertama ( `"Apple"`).
- `fruits[-1]` memberikan elemen terakhir ( `"Mango"`).

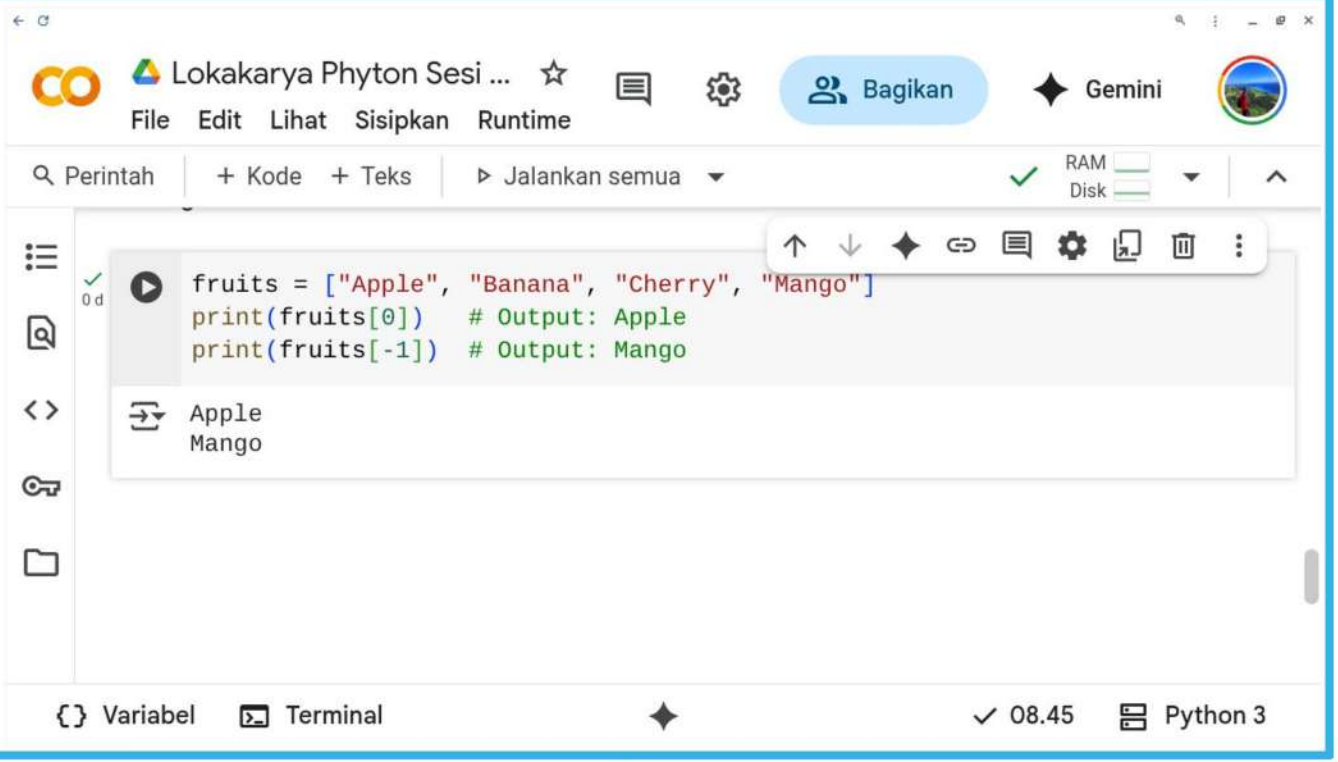
# Membuat Daftar

Mengakses Elemen daftar buah



# Membuat Daftar

Hasil Mengakses Elemen daftar buah



# Mengedit (Memodifikasi) Daftar

Daftar dapat **berubah** , artinya kita dapat mengubah nilainya setelah dibuat.

**Sintaksis:**

```
list_name[index] = new_value
```

**Contoh:**

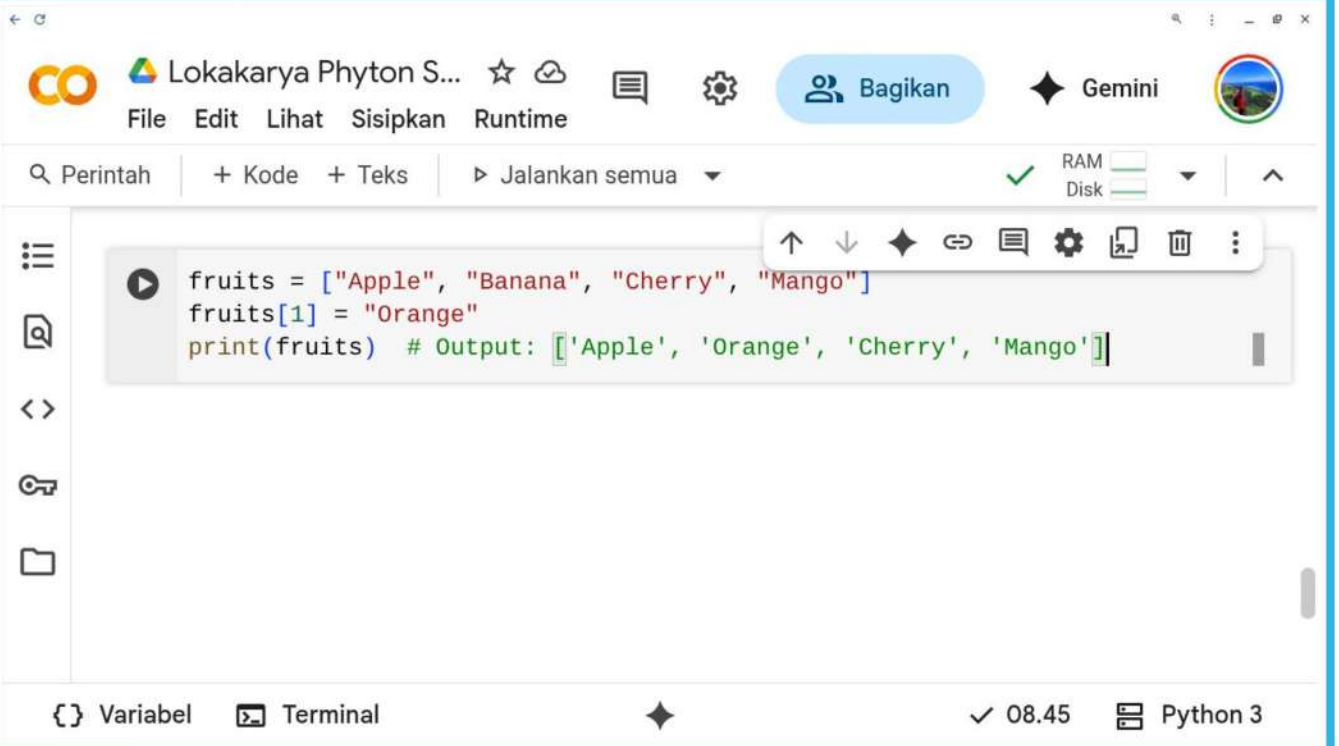
```
fruits = ["Apple", "Banana", "Cherry", "Mango"]
fruits[1] = "Orange"
print(fruits)  # Output: ['Apple', 'Orange', 'Cherry', 'Mango']
```

**Penjelasan:**

- Elemen kedua ( `fruits[1]`) diubah dari `"Banana"`menjadi `"Orange"`.

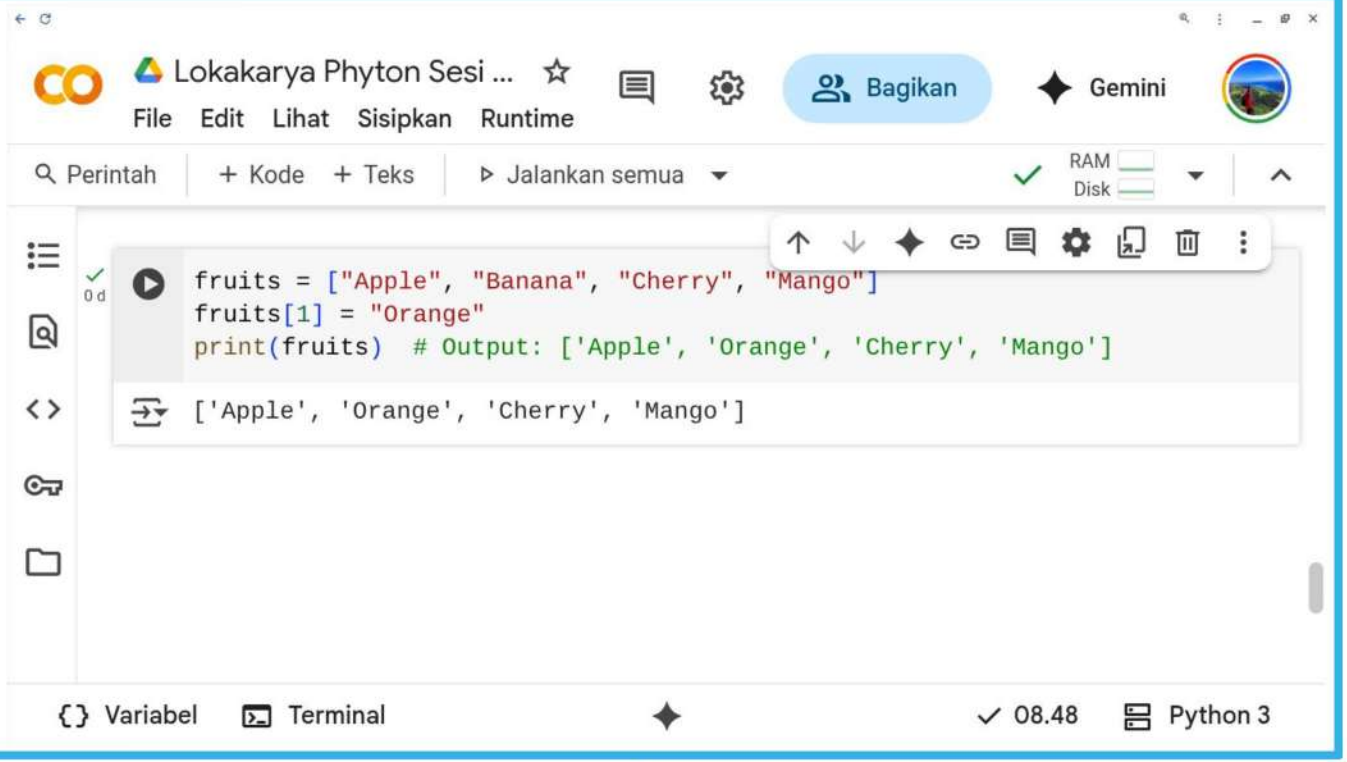
# Mengedit (Memodifikasi) Daftar

Memodifikasi Daftar



# Mengedit (Memodifikasi) Daftar

Hasil Memodifikasi Daftar



# Mengiris Daftar

Pengirisan digunakan untuk mengekstrak **subset** dari suatu daftar. Sintaksnya adalah sebagai berikut `[start:stop:step]`.

**Sintaksis:**

```
list_name[start:stop]  # stop is exclusive
list_name[start:stop:step]
```

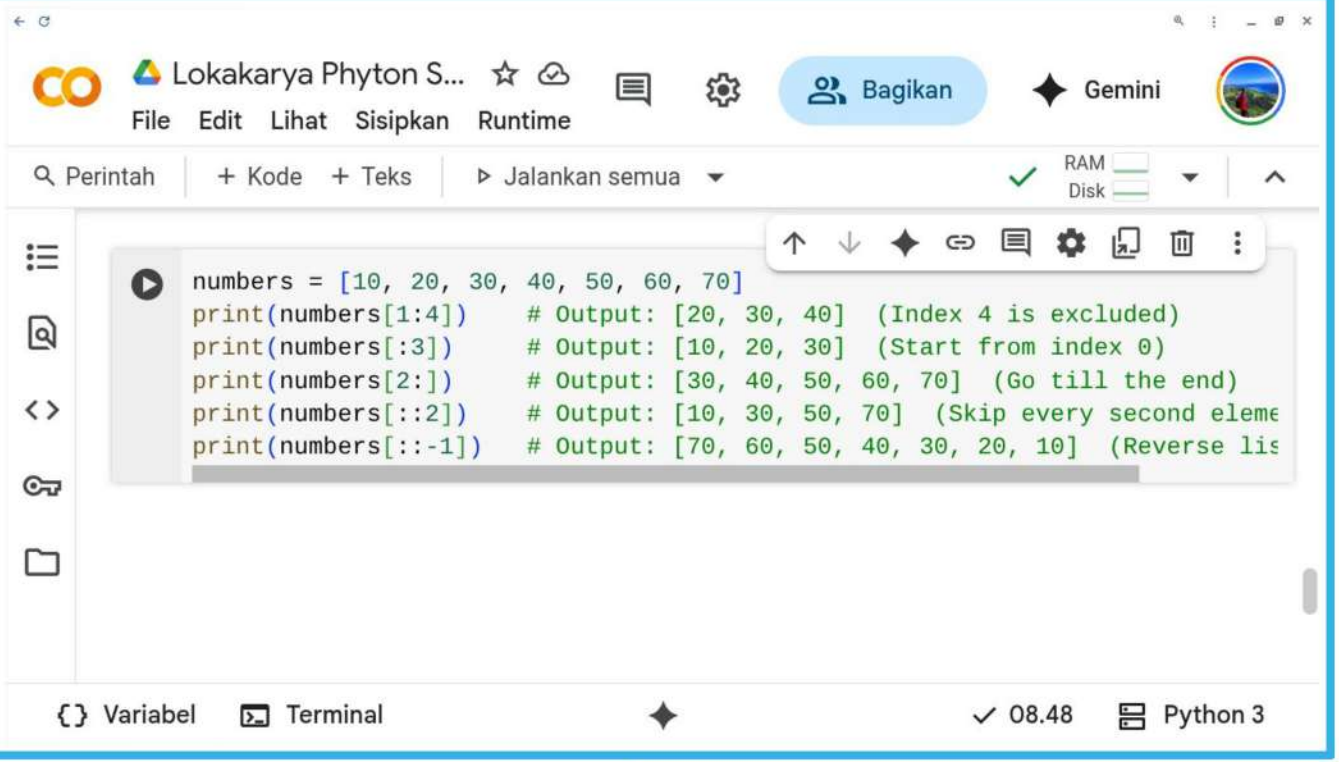
# Mengiris Daftar

**Contoh:**

```
numbers = [10, 20, 30, 40, 50, 60, 70]
print(numbers[1:4])     # Output: [20, 30, 40]  (Index 4 is excluded)
print(numbers[:3])      # Output: [10, 20, 30]  (Start from index 0)
print(numbers[2:])      # Output: [30, 40, 50, 60, 70]  (Go till the end)
print(numbers[::2])     # Output: [10, 30, 50, 70]  (Skip every second element)
print(numbers[::-1])    # Output: [70, 60, 50, 40, 30, 20, 10] (Reverse list)
```

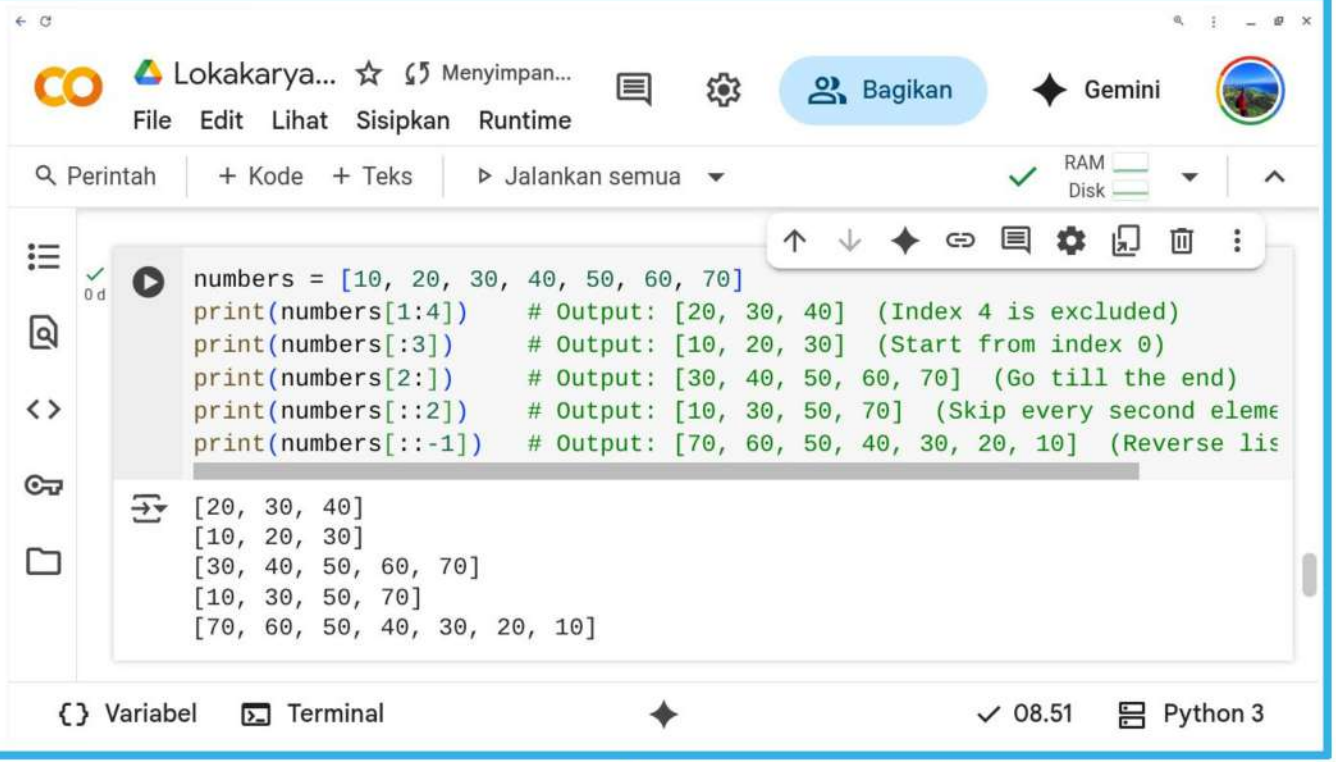
# Mengiris Daftar

Mengiris Daftar Untuk Mengambil Data Tertentu



# Mengiris Daftar

Hasil Mengiris Daftar Untuk Mengambil Data Tertentu



# Mengiris Daftar

**Penjelasan:**

- `[1:4]` memilih elemen dari indeks `1` ke `3`.
- `[:3]` dimulai dari `0` dan berlanjut hingga indeks `2`.
- `[2:]` dimulai dari indeks `2` dan berlanjut sampai akhir.
- `[::2]` melewatkan setiap elemen kedua.
- `[::-1]` membalik daftar.

# Metode Daftar Bawaan

| Metode | Keterangan | Contoh |
|---|---|---|
| `append()` | Menambahkan elemen ke akhir daftar | `fruits.append("Pineapple")` |
| `insert()` | Menyisipkan elemen pada indeks tertentu | `fruits.insert(1, "Grapes")` |
| `remove()` | Menghapus kemunculan pertama elemen yang ditentukan | `fruits.remove("Banana")` |

# Metode Daftar Bawaan

| Metode | Keterangan | Contoh |
|---|---|---|
| `pop()` | Menghapus dan mengembalikan elemen pada indeks tertentu (default terakhir) | `fruits.pop(2)` |
| `index()` | Mengembalikan indeks kemunculan pertama suatu elemen | `fruits.index("Cherry")` |
| `count()` | Mengembalikan jumlah kemunculan suatu elemen | `fruits.count("Mango")` |

# Metode Daftar Bawaan

| Metode | Keterangan | Contoh |
|---|---|---|
| `sort()` | Mengurutkan daftar dalam urutan menaik | `numbers.sort()` |
| `reverse()` | Membalikkan urutan elemen | `fruits.reverse()` |
| `extend()` | Memperluas daftar dengan menambahkan elemen dari daftar lain | `fruits.extend(["Kiwi", "Peach"])` |
| `clear()` | Menghapus semua elemen dari daftar | `fruits.clear()` |

# Metode Daftar Bawaan

**Contoh:**

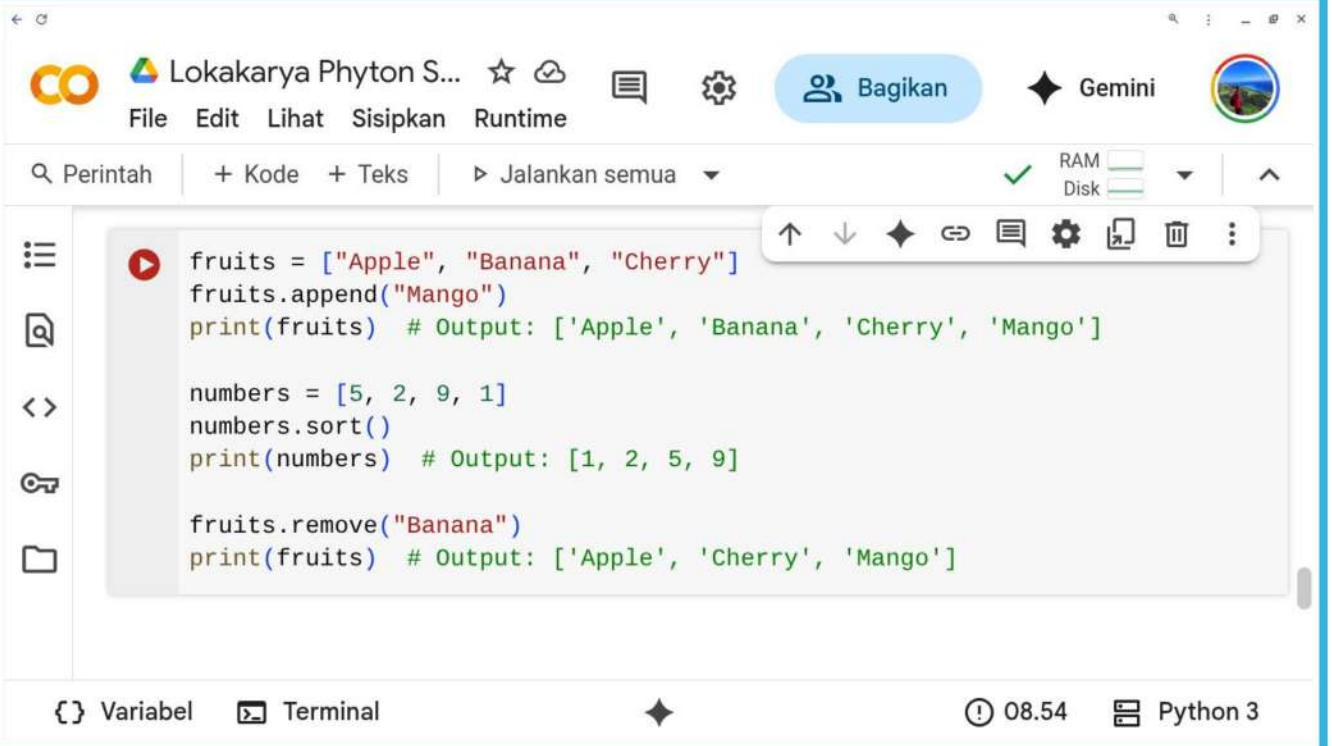
```
fruits = ["Apple", "Banana", "Cherry"]
fruits.append("Mango")
print(fruits)  # Output: ['Apple', 'Banana', 'Cherry', 'Mango']

numbers = [5, 2, 9, 1]
numbers.sort()
print(numbers)  # Output: [1, 2, 5, 9]

fruits.remove("Banana")
print(fruits)  # Output: ['Apple', 'Cherry', 'Mango']
```

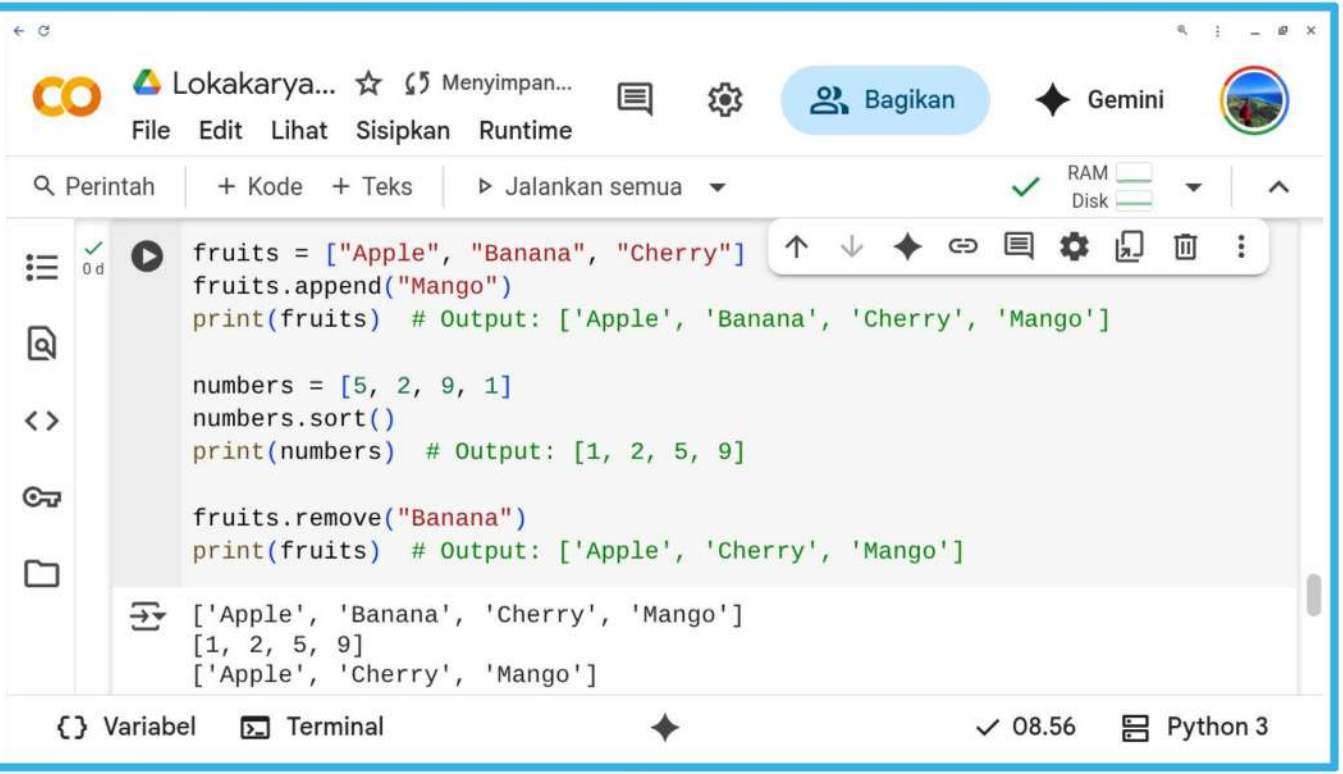
# Metode Daftar Bawaan

Menggunakan Daftar Bawaan



# Metode Daftar Bawaan

Hasil Menggunakan Daftar Bawaan



# Aktivitas: Daftar Tugas Otomatis

**Konsep yang Dicakup:**

- **Menggunakan Daftar** : Menyimpan dan mengelola tugas.
- **While Loop (Forever Loop)** : Menjalankan program secara terus-menerus hingga pengguna keluar.
- **Pernyataan Bersyarat** : Menangani pilihan pengguna yang berbeda.
- **Penanganan Masukan Pengguna** : Memungkinkan pengguna berinteraksi dengan program.

# Aktivitas: Daftar Tugas Otomatis

**1. Membuat Daftar Tugas yang Kosong**

- **Menginisialisasi daftar** kosong untuk menyimpan tugas.

```
todo_list = []
```

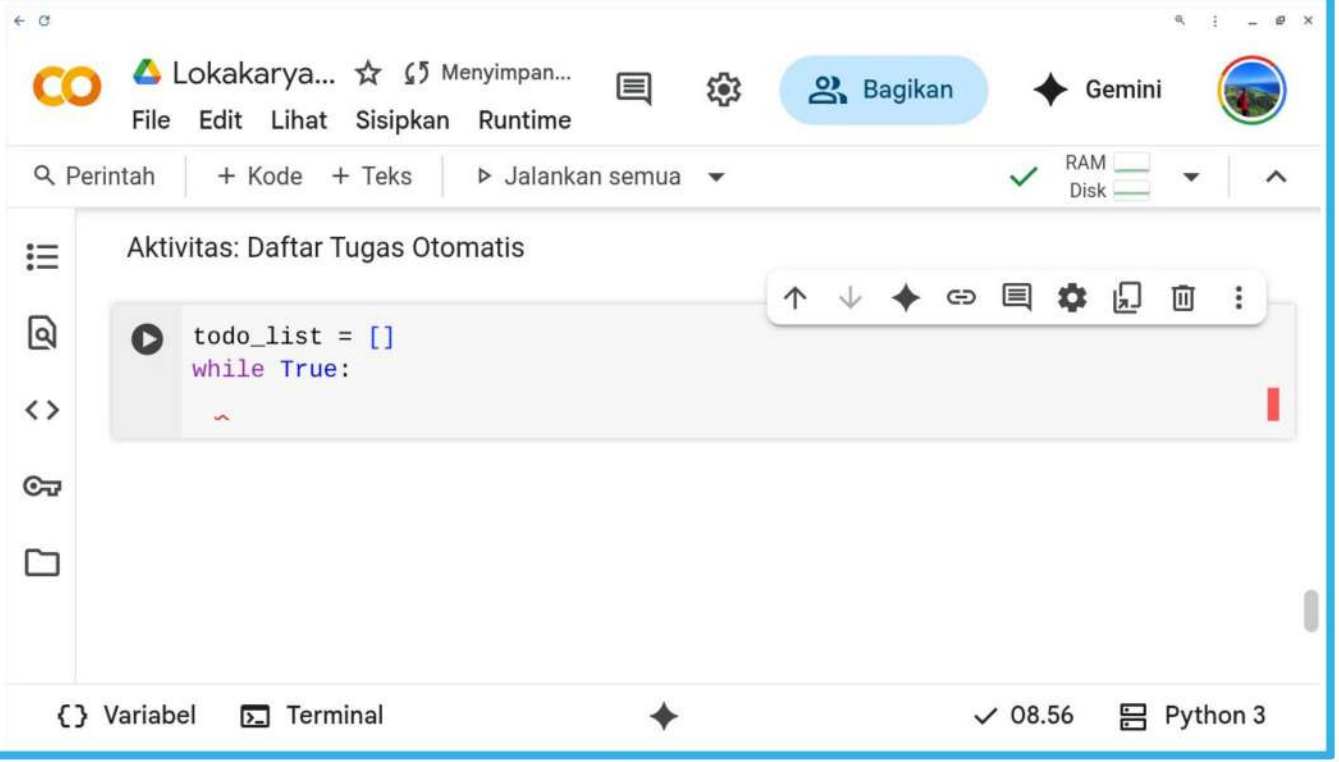
**2. Menjalankan Program dalam Loop Abadi**

- Menggunakan **loop while** untuk menjaga program tetap berjalan **terus menerus** hingga pengguna memilih **Keluar (4)** .

```
while True:
```

# Aktivitas: Daftar Tugas Otomatis

Membuat Daftar Tugas Kosong dan Perulangan Tak Terbatas



# Aktivitas: Daftar Tugas Otomatis

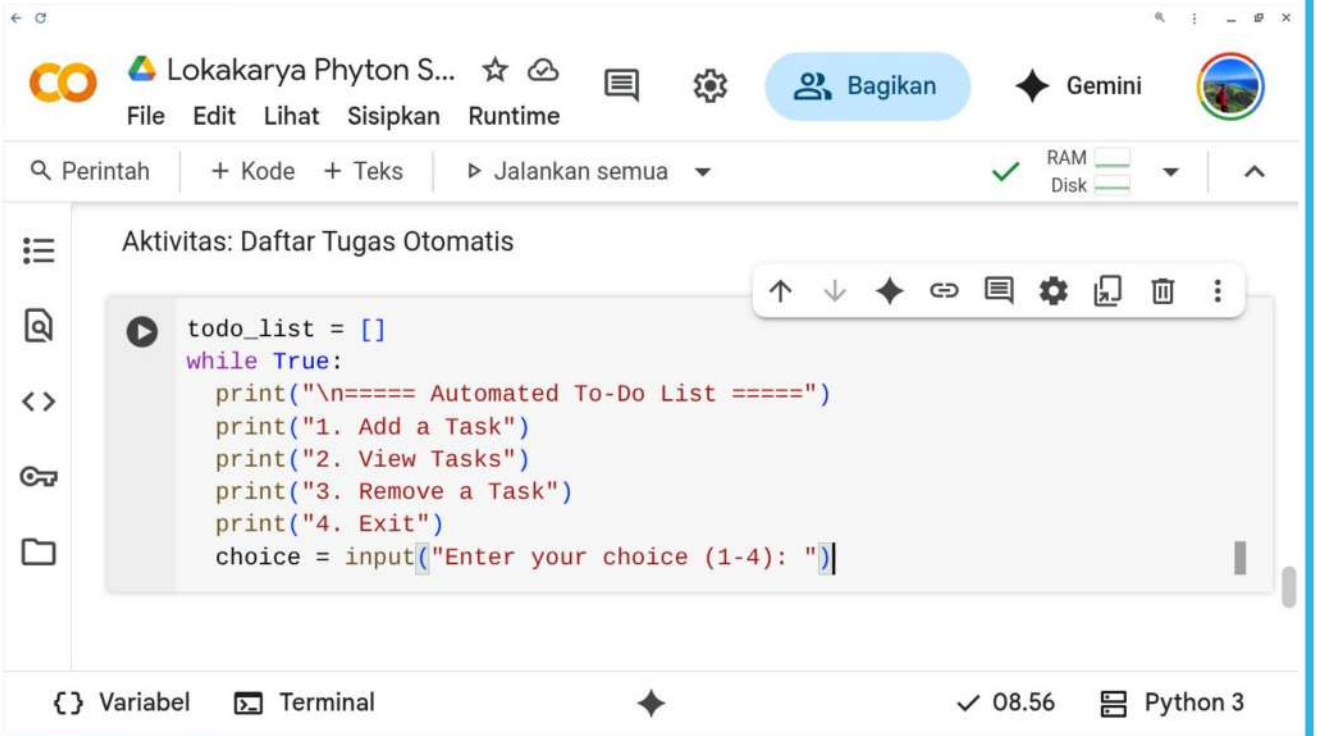
### 3. Menampilkan Opsi Menu

- Menampilkan **menu** dan meminta pengguna untuk **memilih opsi** .

```
print("\n===== Automated To-Do List =====")
print("1. Add a Task")
print("2. View Tasks")
print("3. Remove a Task")
print("4. Exit")
choice = input("Enter your choice (1-4): ")
```

# Aktivitas: Daftar Tugas Otomatis

Cetak Instruksi



# Aktivitas: Daftar Tugas Otomatis

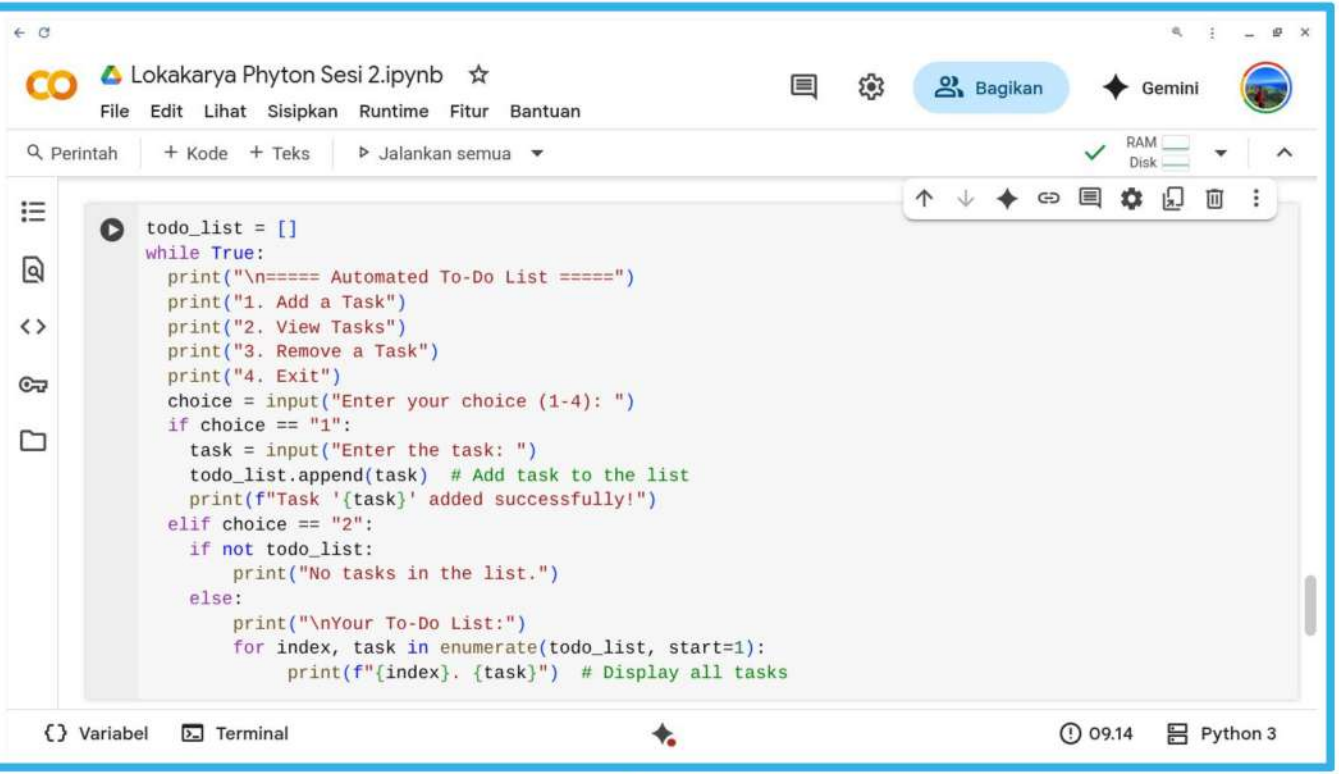
**4. Menambahkan Tugas ke Daftar**

- Pengguna memasukkan **tugas baru** , yang disimpan dalam **todo_list** menggunakan `.append()`.

```
if choice == "1":
    task = input("Enter the task: ")
    todo_list.append(task)  # Add task to the list
        print(f"Task '{task}' added successfully!")
```

# Aktivitas: Daftar Tugas Otomatis

Memasukkan Syarat Perulangan



# Aktivitas: Daftar Tugas Otomatis

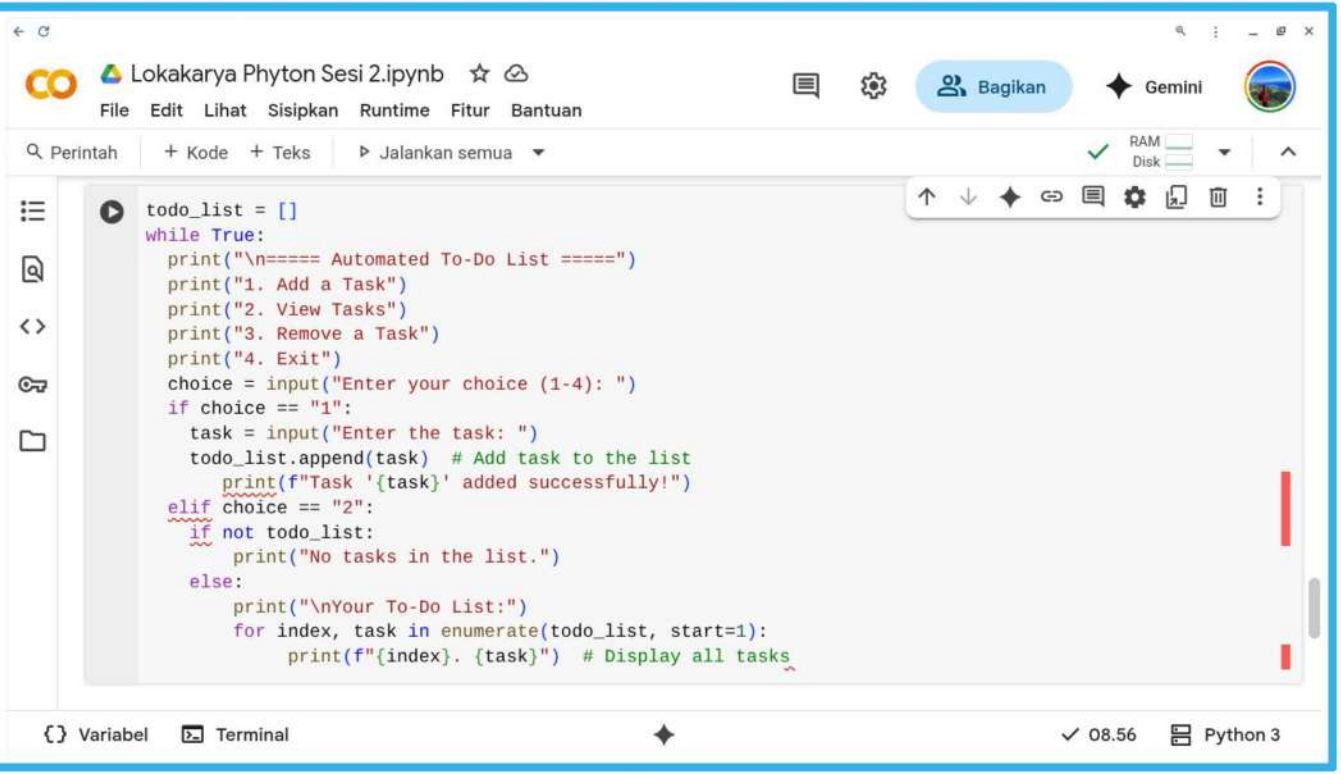
**5. Melihat Tugas**

- Jika daftarnya **kosong** , akan muncul pesan.
- Jika tidak, ia **akan mengulang** daftar dan menampilkan tugas **dengan angka** .

```
elif choice == "2":
    if not todo_list:
        print("No tasks in the list.")
    else:
        print("\nYour To-Do List:")
        for index, task in enumerate(todo_list, start=1):
            print(f"{index}. {task}")  # Display all tasks
```

# Aktivitas: Daftar Tugas Otomatis

Memasukkan Syarat Perulangan



# Aktivitas: Daftar Tugas Otomatis

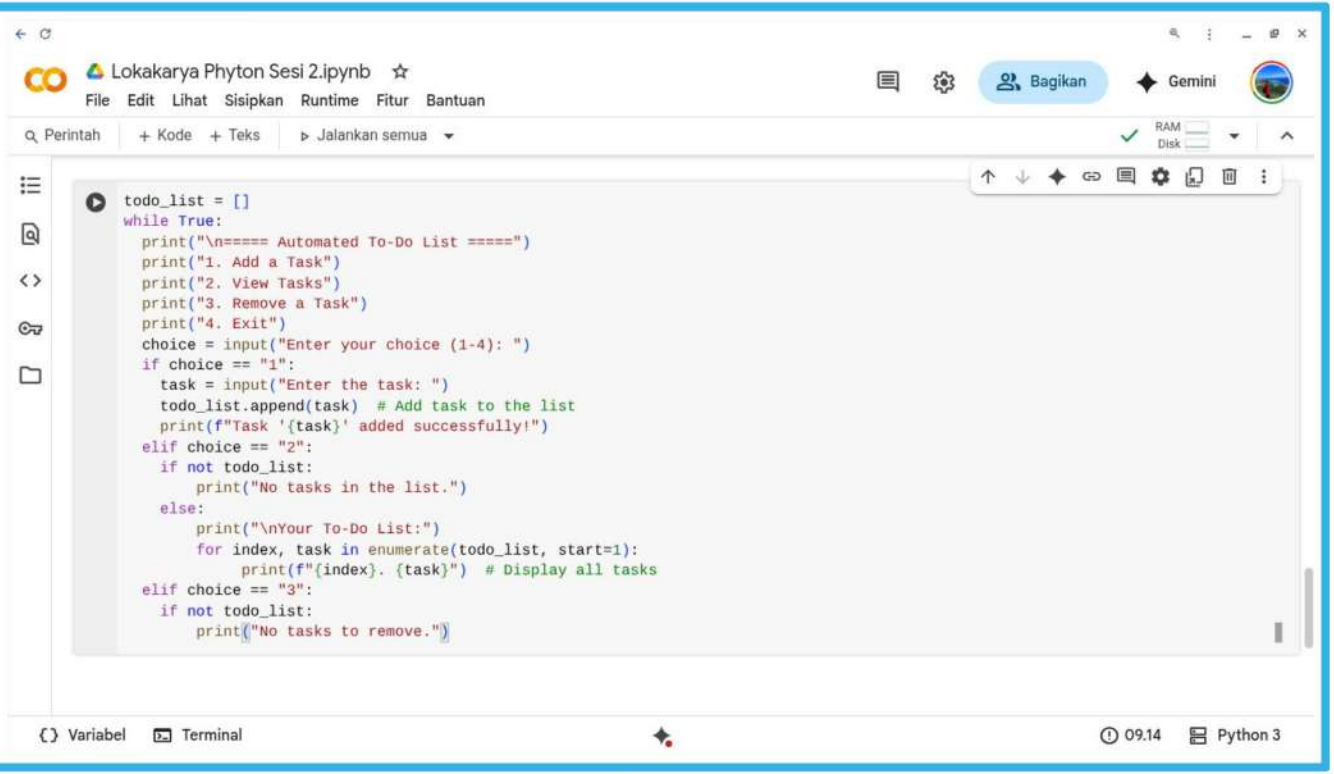
**6. Menghapus Tugas**

- Pengguna memasukkan **nomor tugas** yang ingin dihapus.
- Tugas dihapus dari **daftar** menggunakan `.pop()`.

```
elif choice == "3":
    if not todo_list:
        print("No tasks to remove.")
```

# Aktivitas: Daftar Tugas Otomatis

Memasukkan Syarat Perulangan

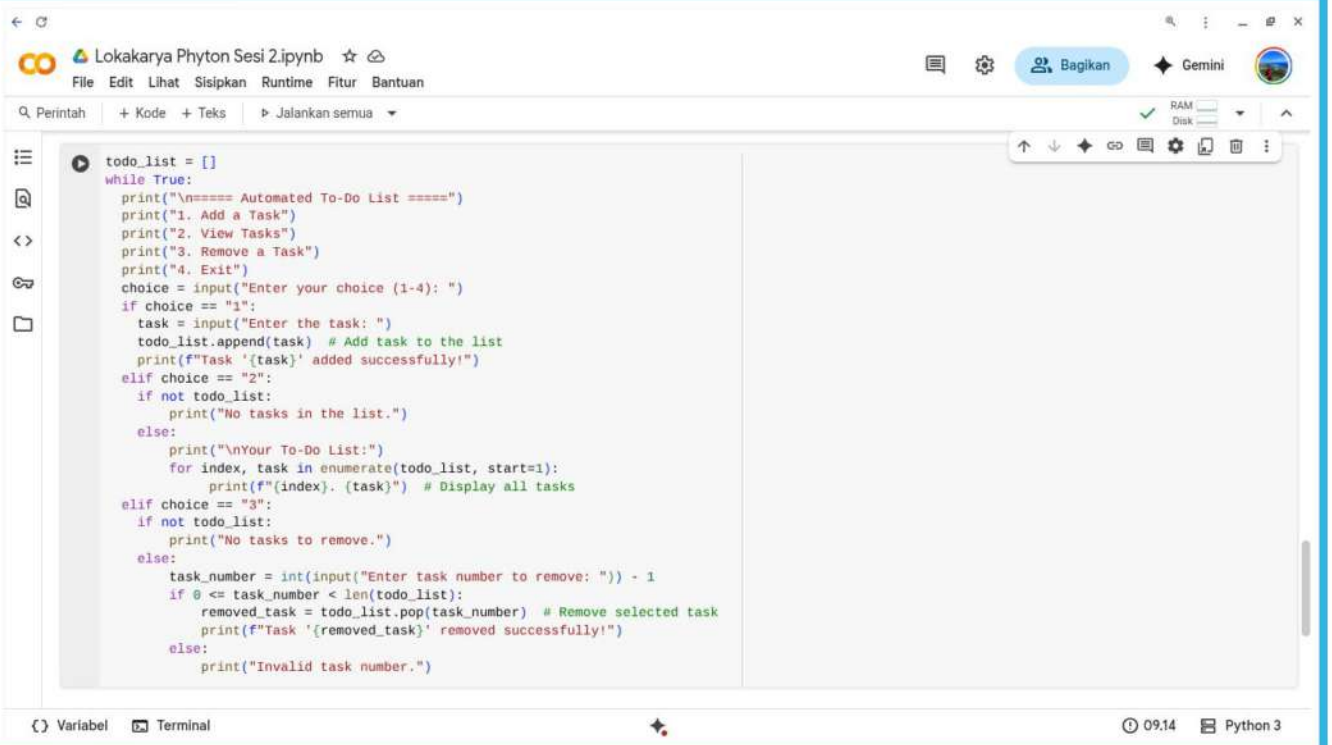


# Aktivitas: Daftar Tugas Otomatis

```
    else:
        task_number = int(input("Enter task number to
remove: ")) - 1
        if 0 <= task_number < len(todo_list):
            removed_task = todo_list.pop(task_number)   #
Remove selected task
            print(f"Task '{removed_task}' removed
successfully!")
        else:
            print("Invalid task number.")
```

# Aktivitas: Daftar Tugas Otomatis

Memasukkan Syarat Perulangan




```
todo_list = []
while True:
  print("\n===== Automated To-Do List =====")
  print("1. Add a Task")
  print("2. View Tasks")
  print("3. Remove a Task")
  print("4. Exit")
  choice = input("Enter your choice (1-4): ")
  if choice == "1":
    task = input("Enter the task: ")
    todo_list.append(task)  # Add task to the list
    print(f"Task '{task}' added successfully!")
  elif choice == "2":
    if not todo_list:
        print("No tasks in the list.")
    else:
        print("\nYour To-Do List:")
        for index, task in enumerate(todo_list, start=1):
            print(f"{index}. {task}")  # Display all tasks
  elif choice == "3":
    if not todo_list:
        print("No tasks to remove.")
    else:
        task_number = int(input("Enter task number to remove: ")) - 1
        if 0 <= task_number < len(todo_list):
            removed_task = todo_list.pop(task_number)  # Remove selected task
            print(f"Task '{removed_task}' removed successfully!")
        else:
            print("Invalid task number.")
```

# Aktivitas: Daftar Tugas Otomatis

**7. Keluar dari Program**

- Jika pengguna **memilih 4** , program berhenti menggunakan `break`.

```
elif choice == "4":
    print("Exiting To-Do List. Have a productive day!")
    break  # Exit the loop
```

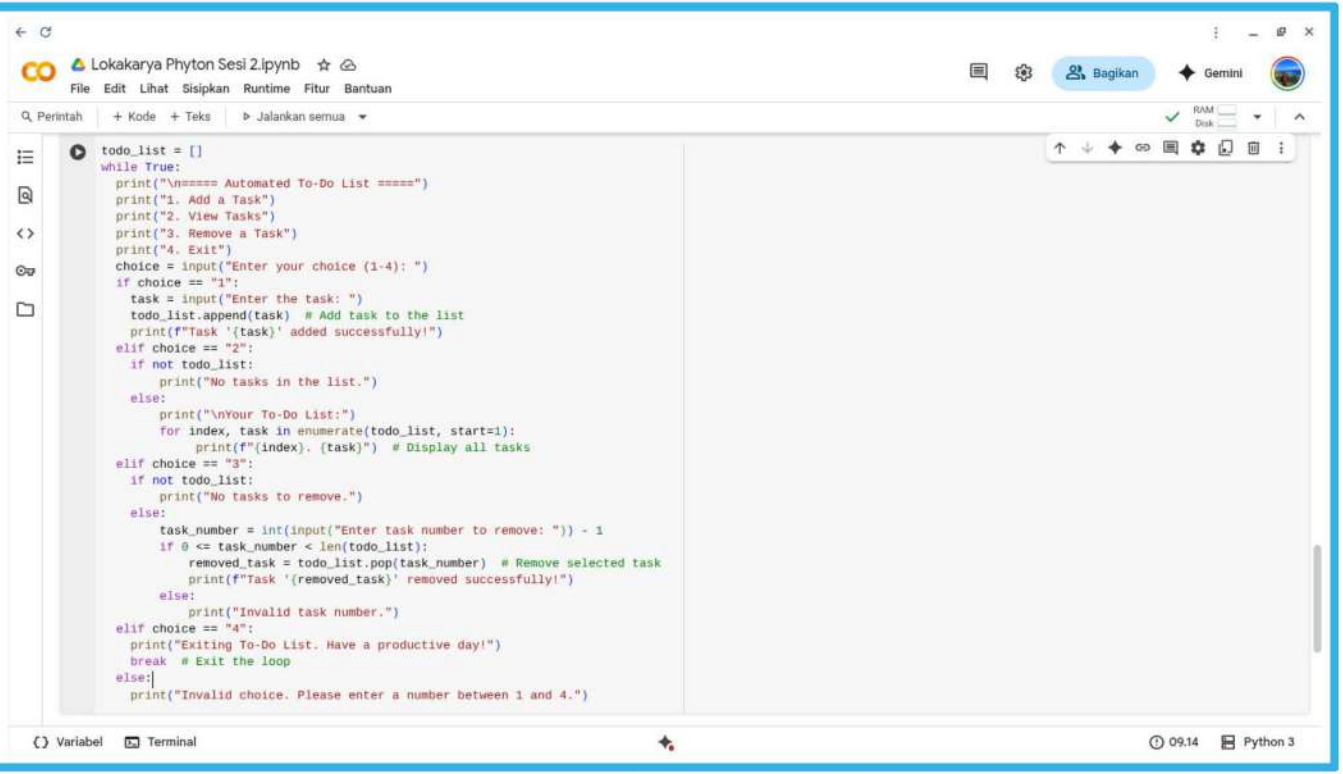
**8. Menangani Input yang Tidak Valid**

- Memastikan pengguna **hanya memasukkan pilihan yang valid** (1-4).

```
else:
    print("Invalid choice. Please enter a number between 1
          and 4.")
```

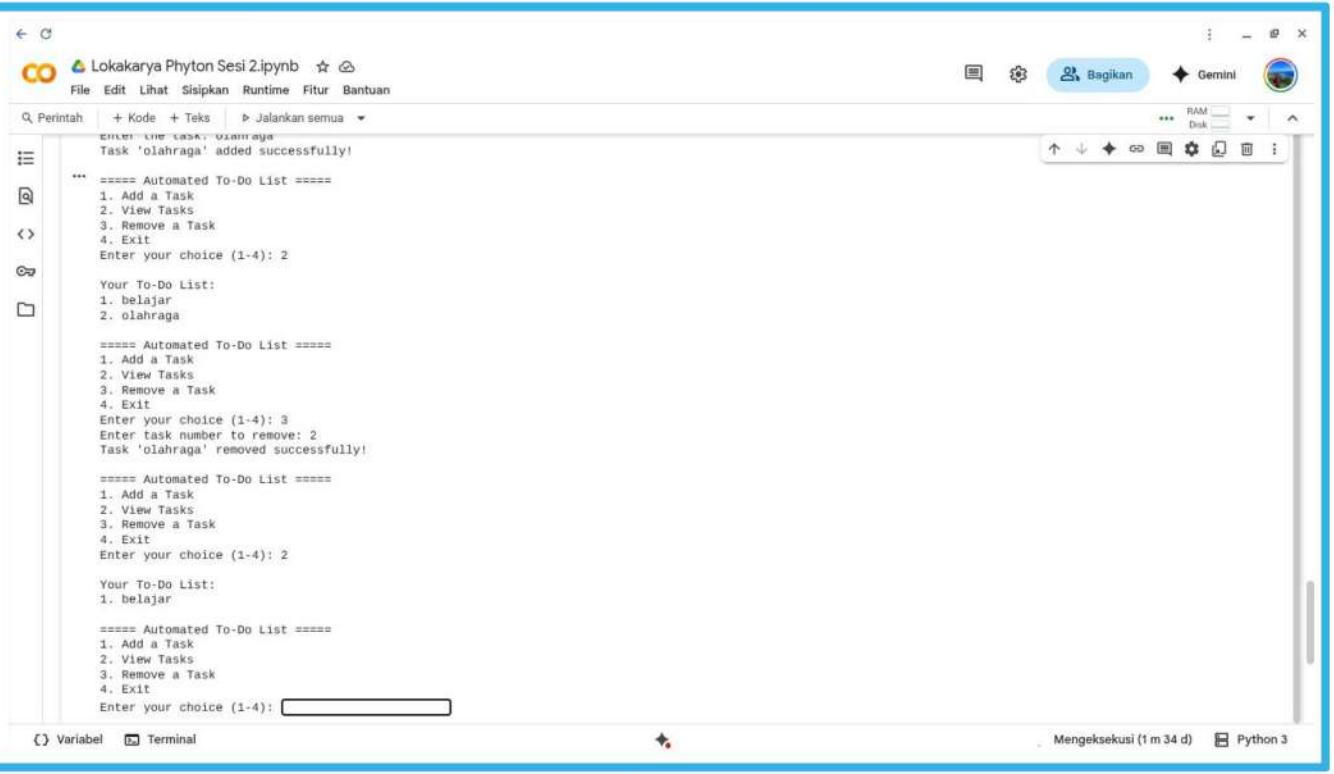
# Aktivitas: Daftar Tugas Otomatis

Memasukkan Syarat Perulangan



# Aktivitas: Daftar Tugas Otomatis

Masukkan sesuai instruksi



# Tantangan Phyton

1. **Membuat Tabel Perkalian**

   Gunakan loop untuk mencetak tabel perkalian untuk angka tertentu (dari 1 hingga 10).

2. **Jumlah Semua Angka dalam Daftar**

   Tulis program yang menghitung jumlah semua angka dalam daftar menggunakan perulangan.

3. **Periksa apakah Daftar Berisi Item Tertentu**

   Buat program yang meminta pengguna untuk memberikan daftar item, lalu memeriksa apakah item tertentu ada dalam daftar tersebut.

4. **Perulangan Bersarang: Tabel Perkalian**

   Gunakan perulangan bersarang untuk menghasilkan dan menampilkan tabel perkalian dari 1 hingga 1